STRIPE PROJECT

4. Data Pipeline

Annexes

4.B. Feature Store Notes

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D5 : Data Pipeline Architecture / Annexe B / v10 --- 2026/11/05.*

4.B.1. Le besoin : deux magasins de features distincts

Le système de scoring de fraude avec entrainement continu a besoin de deux magasins de features distincts :

- **Un magasin en ligne** pour servir l'*inférence en production* :
  - au moment où une transaction arrive, le service de scoring lit l'état courant du client dans le magasin en ligne (une seule lecture par clé, en quelques millisecondes, sans jointure). L'état courant contient le dernier calcul de scoring, et c'est tout ([D4] §D4.5.4 : *"le magasin de features NoSQL est un cache d'état courant"*) ;
  - implementation : *MongoDB* / `ml_features` ([réf.?]).
- **Un magasin hors ligne** pour servir l'*entraînement* :
  - pour ré-entraîner ou valider un modèle, il faut l'historique complet de chaque feature à travers le temps, interrogé par des requêtes analytiques massives, des agrégations, des jointures temporelles (cf. [D7, §D7.4.5]). L'état courant seul ne suffit plus : on a besoin de savoir ce qu'était une certaine propriété dans le passé (ex: il y a six mois), pour reconstituer les conditions d'une transaction passée ;
  - implementation : *Snowflake* + schéma dédié ([réf.?]).

Un seul magasin ne peut pas répondre aux deux besoins en même temps

Un système optimisé pour l'un est structurellement mauvais pour l'autre :

- *MongoDB*, excellent pour une lecture par clé en millisecondes, serait catastrophique pour scanner des mois d'historique et faire des agrégations analytiques lourdes.
- *Snowflake*, excellent pour ce type d'analyse, introduirait une latence bien trop élevée pour une décision de fraude qui doit tomber en moins de 100 ms.

4.B.2. Le risque : décalage entre entraînement et service

Le risque de devoir gérer deux magasins de features en même temps est le *décalage entre entraînement et service* (risque déjà identifié dans [D7, §D7.4.6]) : un modèle entraîné sur des features calculées d'une certaine manière, puis interrogé en production avec des features calculées d'une autre manière, produit des résultats faux sans qu'aucune erreur ne se déclenche.

Séparer les deux magasins *tout en gardant une seule source de calcul* garantit que les deux systèmes de stockage sont cohérents entre eux.

4.B.3. La solution : orchestration centralisée des features

Dans une architecture classique, l'orchestration des features garantie que la même définition de feature alimente les deux magasins malgré les différence de forme et de vitesse de traitement :

	Magasin online	**Magasin offline**
Contenu	État courant seulement	Historique complet
Accès	Lecture par clé, <10ms	Requêtes analytiques, minutes
Technologie	*MongoDB*	*Snowflake*
Service	scoring	entraînement

Aperçu de la plateforme *Feast*. Le fonctionnement d'un orchestrateur comme *Feast* [réf.?] est, d'une part, de conserver les définitions uniques des features dans un registre central, d'autre part, de matérialiser les features ainsi définis, à la fois dans le magasin en ligne et dans le magasin hors ligne. C'est une sorte de plateforme de gouvernance qui empêche mécaniquement la duplication des définitions logiques.

Alternative *Flink*. Dans l'architecture proposée [D1], on a déjà un mécanisme simple qui joue le rôle d'orchestration : un unique job *Flink* calcule et actualise chaque feature du mod_7èle (§D7.3.3) et écrit directement le résultat dans les deux magasins (ici, le job *Flink* joue donc également le rôle de registre des définitions des features).

Conditions de passage à *Feast*. Cette plateforme a un intérêt quand plusieurs modèles, plusieurs équipes partagent des définitions de features et ont besoin d'un catalogue commun pour éviter que chacun recalcule la même chose à sa façon. Dans le projet *Sprite*, on ne considère qu'un seul modèle (le scoring de fraude) et une seule équipe de calcul (le job *Flink*).

Le passage à une plateforme comme *Feast* serait justifié si un modèle de recommandation ou de prédiction de désabonnement était mis en place tout en partageant certains des mêmes features que la fraude.

Références

Spécifications

- [D0] *Stripe Project Introduction*

Livrables

- [D1] *Comprehensive Data Architecture Diagram*
- [D4]
- [D5] *Data Pipeline Architecture*
- [D7] *Machine Learning Integration Strategy*

Document Applicables

[réf.?][*Feast*][]